The use of symbolic addressing allows a programmer to write some assembly language code without having to bother about where the code will be stored in memory when the program is run. However, it is possible to write assembly language code where the symbolic addressing is replaced by either relative addressing or absolute addressing. Table 6.02 shows the simple code from Table 6.01 converted to use these alternative approaches.

Assembly language program using relative addressing		Assembly language program using absolute addressing	
(0)	IN	(200)	IN
(1)	SUB #48	(201)	SUB #48
(2)	STO [BR] + 15	(202)	STO 215
(3)	LDM #0	(203)	LDM #0
(4)	STO [BR] + 16	(204)	STO 216
(5)	STO [BR] + 17	(205)	STO 217
(6)	IN	(206)	IN
(7)	SUB #48	(207)	SUB #48
(8)	ADD [BR] + 16	(208)	ADD 216
(9)	STO [BR] + 16	(209)	STO 216
(10)	LDD [BR] + 17	(210)	LDD 217
(11)	INC ACC	(211)	INC ACC
(12)	CMP [BR] + 15	(212)	CMP 215
(13)	JPN [BR] + 7	(213)	JPN 207
(14)	END	(214)	END
(15)		(215)	
(16)		(216)	
(17)		(217)	

Table 6.02 A simple assembly language program using relative and absolute addressing

For the relative addressing example, the assumption is that a special-function base register BR contains the base address. The contents of this register can then be used as indicated by [BR]. Note that there are no labels for the code. The left-hand column is just for illustration identifying the offset from the base address which is the address of the first instruction in the program.

For the absolute address example there are again no labels for the code. The left-hand column is again just for illustration but this time identifying actual memory addresses. This has been coded with the understanding that the first instruction in the program is to be stored at memory address 200.

6.04 The assembly process for a two-pass assembler

For any assembler there are a number of things that have to be done with the assembly language code before any translation can be done. Some examples are:

- removal of comments
- replacement of a macro name used in an instruction by the list of instructions that constitute the macro definition
- removal and storage of directives to be acted upon later.

A two-pass assembler is designed to handle programs written in the style of the one illustrated in Table 6.01. This program contains forward references. Some of the instructions have a symbolic address for the operand where the location of the address is not known at that stage of the program. A two-pass assembler is needed so that in the first pass the location of the addresses for forward references can be identified.

To achieve this during the first pass the assembler uses a symbol table. The code is read line by line. When a symbolic address is met for the first time its name is entered into the symbol table. Alongside the name a corresponding address has to be added as soon as that can be identified. Table 6.03 shows a possible format for the symbol table that would be created for the program shown in Table 6.01.

Symbol	Offset
MAX	+15
TOTAL	+16
COUNT	+17
STRTLP	+7

Table 6.03 A completed symbol table for the assembly language program in Table 6.01

Note that the assembler has to count the instructions as it reads the code. Then when it encounters a label it can enter the offset value into the symbol table. In this example the first entry made in the offset column is the +7 for `STRPLP`.

For the second pass the Assembler uses the symbol table and a lookup table that contains the binary code for each opcode. This table would have an entry for every opcode in the set defined for the processor. Table 6.04 shows entries only for the instructions used in the simple program we are using as an example. Note that the binary codes are just suggestions of codes that might be used.

Opcode mnemonic	Opcode binary
IN	0001 0000
SUB	0110 0001
STO	0100 0100
LDM	0010 0001
ADD	0100 0101
LDD	0010 0101
INC	0101 0101
CMP	1000 0100
JPN	1010 0100
END	1111 1111

Table 6.04 An opcode lookup table

Provided that no errors have been identified, the output from the second pass will be a machine code program. For our example, this code is shown in Table 6.05 along with the original assembly code for comparison.

Machine code		Assembly code	
Opcode	Operand		
0001 0000			IN
0110 0001 0000 0000 0011 0000			SUB #48
0100 0100 0000 0000 0000 1111			STO MAX
0010 0001 0000 0000 0000 0000			LDM #0
0100 0100 0000 0000 0001 0000			STO TOTAL
0100 0100 0000 0000 0001 0001			STO COUNT
0001 0000		STRTLP:	IN
0110 0001 0000 0000 0011 0000			SUB #48
0100 0101 0000 0000 0001 0000			ADD TOTAL
0100 0100 0000 0000 0001 0000			STO TOTAL
0010 0101 0000 0000 0001 0001			LDD COUNT
0101 0101			INC ACC
1000 0100 0000 0000 0000 1111			CMP MAX
1010 0100 0000 0000 0000 0110			JPN STRTLP
1111 1111			END
0000 0000		MAX:	
0000 0000		TOTAL:	
0000 0000		COUNT:	

Table 6.05 Machine code created from assembly code

Some points to note are as follows.

- Most of the instructions have an operand which is a 16-bit binary number.
- Usually this represents an address but for the `SUB` and `LDM` instructions the operand is used as a value.
- There is no operand for the `IN` and `END` instructions.
- The `INC` instruction is a special case. There is an operand in the assembly language code but this just identifies a register. In the machine code the register is identified within the opcode so no operand is needed.
- The machine code has been coded with the first instruction occupying address zero.
- This code is not executable in this form but it is valid output from the assembler.
- Changes will be needed for the addresses when the program is loaded into memory ready for it to be executed.
- Three memory locations following the program code have been allocated a value zero to ensure that they are available for use by the program when it is executed.

6.05 Addressing modes

When an instruction requires a value to be loaded into a register there are different ways of identifying the value. Each one is known as an **addressing mode**. In Section 6.01, it was stated that, for our simple processor, two bits of the opcode in a machine code instruction would be used to define the addressing mode. This allows four different modes which are described in Table 6.06.



KEY TERM

Addressing mode: when the instruction uses a value this defines how the operand must be used to find the value